MMA3001 – Numerical Methods and Machine Learning

Automated Packaging-Defect Inspection of Pork Rasher Trays with YOLOv8

Zhafira Ayu Sabrina | 33704228

1.0 Engineering Problem

Automated visual inspection is one of the most common uses of machine learning in manufacturing. Every pack leaving a food-packaging line must be checked for seal integrity and presentation, but manual inspection is limited by fatigue, inconsistent judgement and line speed. A camera paired with a deep-learning object detector can instead inspect every pack and log each result automatically.

This project applies the convolutional object detector YOLOv8 [1] to images of packaged pork-rasher trays. It detects five defects (unsealed film edge, general packaging error, loose meat, twisted meat and wrinkled film) and decides whether each tray passes or is rejected. These defects shorten shelf life, create food-safety risks and can lead to complaints or recalls if a pack reaches a customer, whereas a false rejection only costs a manual re-check. Missing a defect is therefore the more costly error, and the decision rule is designed around this. The system is intended as a decision-support tool at the end of the line: it sets aside suspect trays for an operator and records each defect's type, position and severity.

The engineering question is whether the model can be trusted on trays it has never seen. The objectives are to define the cost of each type of error; to build a detection and decision pipeline using transfer learning with YOLOv8n; to compare it with an "always FAIL" rule, a classical baseline and a larger YOLOv8s model; to validate it on data independent of training; and to analyse its sensitivity to resolution, robustness, speed, memory and computational cost.

Validation is the main focus. The image file names revealed that the dataset consists of consecutive video frames, so the official test set contains near-duplicates of training frames. On a leakage-free split built from separate video sequences, the detector caught 94.8% of defective trays, but its balanced accuracy was 0.688, compared with 0.873 on the official split (Section 5).

The scope is limited to one top-down camera, the lighting and trays in the dataset, and a single product. Wrinkle (18 training examples) and loose meat (66) are too rare to evaluate reliably, and testing was done offline on recorded frames.

2.0 Inputs, Outputs and Data

2.1 Top-Level inputs and outputs

Input. An RGB image (JPG/PNG, 8-bit per channel, 0-255) of one tray taken from above, 720 x 540 px image; other sizes are acceptable, however are flagged as outside the set domain.

Outputs. The outputs in this model include a list of detections, where it comes from each a class (loose-meat, package-error, twisted-meat, unsealed, wrinkle), a confidence from 0 to 1 and box corners in pixels, showing to the operator what and where each specific defect is. Another output the model will give would be a set PASS/FAIL decision, where FAIL occurs when any confidence is a minimum of $\tau = 0.10$. Additionally, it will output a grade based on how severe the defect is from the fraction of the image (where <1% minor, 1-5% major, >5% critical), as it prioritises re-checks. Lastly, a CSV/JSON report with an annotated image will be provided as an audit trail. The full specifications can be found in Appendix C.

2.2 Dataset

The Pork Rasher Error (Packaging) v4 dataset [2] (Roboflow Universe, CC BY 4.0) contains 3,549 annotated images in YOLO format: one text file per image, with one line per box (class, x-centre, y-centre, width, height, normalised to 0–1). An empty label file denotes a defect-free tray. The official split has 3,349 training, 120 validation and 80 test images; per-split counts are in Appendix C.

The data has three properties that determine every later decision. Class imbalance, where packaging-error and unsealed make up 95% of the training boxes; wrinkle has 18 and 0 in the official validation or test sets, therefore, cannot be evaluated properly (can be found in Appendix B). Furthermore, small objects are another property, with the median packaging-error box covering about 0.35% of the image, versus about 11% for the unsealed (found in Appendix B). Small objects are generally harder for the model to detect, and sensitive to input resolution. Additionally, most trays are defective, with 80% of images having a defect, therefore, this can cause a mislead for the plain accuracy. Always answering FAIL already scores about 80%, that is why balance accuracy is applied, which will be further discussed in section 5.

2.3 Data Leakage in the Official Split

The image file names show that the images are frames of a single production video. The 3,349 training images contain only 1,085 distinct frames, because Roboflow exported about three brightness-augmented copies of each. Since neighbouring frames are nearly identical, the gap from every validation and test image to the nearest training frame was measured. On the official split, 4 test frames are exact duplicates of training frames, the median gap is only 2.5 frames, and 67.5% of test frames lie within 5 frames of a training frame [3] (Table 3, Section 5).

3.0 Computation solution

The computation solution consists of a two-stage pipeline, with a YOLOv8n detector finding defects, and a simple, transparent decision layer turns the detections into a PASS / FAIL output for each tray.

3.1 Method

YOLOv8n is essentially a single-stage convolutional object detector that originates from the YOLO (“You Only Look Once”) family [4]. The method consists of three parts: a backbone of convolution layers is used to extract features at different scales; a neck takes into account for both small and large objects by combining coarse and fine feature maps, and; a detection head where it can predict for every feature map, a box, a confidence and a class score at all locations.

Overlapping boxes usually are proposed by the network for the same type of defect.To choose and pick between the overlapping boxes, it measures how much the two boxes intersect over each other through the use of Intersection over Union (IoU), where the shared area is divided by the total area that they cover. The equation is displayed below.

$$IoU(A, B) = |A \cap B| \div |A \cup B|$$

Non-maximum suppression allows for any overlapping duplicate boxes to be removed in order for one forward pass to yield all detections for an image by keeping the most confident box, and any other box that intersects it with an IoU of > 0.7 would be removed. The same procedure applies when it comes to evaluation: a prediction is determined correct when the IoU with a true box of the same class is at least 0.5

Through the process of training, it allows adjustments to be made on the network's weights by gradient descent, using AdamW optimiser with a learning rate of 0.0011 (chosen automatically by Ultralytics. The minimised loss accounts for how far the predicted boxes are from the true boxes, and how incorrect the predicted classes are. Moreover, instead of starting with randomised weights, the model applies transfer learning. The method begins from pre-trained weights on COCO [5], a (public) dataset consisting of approx. 330,000 everyday photos in 80 classes, and only the last layer is taken and replaced in order to predict the five defect classes. This enables the model to develop the skill to recognise general shapes and edges, and would only need to learn what the defects look like. Therefore, about a thousand distinct video frames are sufficient for this method.

To minimise more over-fitting, every image under training is flipped randomly, scaled, shifted and modified in colour and brightness. For a majority of the epochs, four images are further pieced together into a mosaic. These augmented images show the model newer versions of the same trays so that the model can learn the defect itself rather than just memorising the particular image. Piecing images together into mosaics stops when it approaches the last ten epochs, so that the model can complete the training run on the real images. The decision layer is then applied so that each tray is given a score equal to the most confident detection, and would fail the tray if the score reaches the threshold, τ, chosen on the validation set (Section 5.2):

$$\textbf{score(tray)} = \max_i c_i, \qquad \textbf{FAIL} \Leftrightarrow \textbf{score(tray)} \geq \tau$$

Where c_i represents the confidence of detection i. A grade showing the severity is also determined by getting the total area of the detected boxes and dividing by the image area.

3.2 Why the Method was Selected

The object-detection approach was selected over a simple good/defective classifier, and YOLOv8n was chosen as the detector for the production line due to its ability to fulfill the requirements. There were also 6 main reasons why this method was chosen:

- Speed:Only requires one pass through the network per image and takes around 9ms to do so. This speed is fast enough for a machine that moves at high speeds along a conveyor belt.
- Localisation: Unlike a classifier that can only tell if an object is defective, the detector can show the operator where the defect is located on the product.
- Small data: Transferlearning makes it practical with this small dataset.
- Hardware: The nano version of the model has only about 3.1 million parameters (6.5 MB), which means it can be trained and run on a lower GPU laptop, even a small edge device.
- Data format: The dataset was supplied in the YOLO format.
- Builds on the unit: The project builds upon the unit, specifically neural networks and regression [6]. The project utilises the same technique for training a deep neural network, such as early stopping, seeded initialisation of the weights and checking the count of parameters within the network. The project is also validated using methods from the unit notes on selecting fair training and test data and on measuring model performance.

3.3 Settings and Parameters

The model training was done on 640-pixel images in batches of 8 for the majority of the 30 epochs. Training however, was stopped early if the validation mAP did not show any improvement after 10 epochs to prevent wasted time and over-fitting. The default Ultralytics augmentation and optimiser settings were utilised and the random seed was kept at 42 in deterministic mode to allow for reproducibility of the results. The decision threshold, $\boldsymbol{\tau = 0.10,}$ is the highest value that still catches at least 95% of defective trays in the

validation set. Since a missed defect would result in higher costs than a false alarm, recall was given priority over precision. All of these settings are kept and stored in a single configuration file, and listed out in Appendix C with the corresponding justifications.

3.4 Assumptions

The approach depends on four assumptions:

- The camera, lighting and trays in the deployed version of the system match those in the video
- The annotations are correct and complete, since an unlabelled defect is considered a good tray. Any annotated defect, however small, makes the tray defective
- Frames that are more than 20 apart will display different trays, which are trays that the grouped split in Section 5 rely on.

3.5 Implementation

The code is organised as a Python package (src/porkqc) that holds separate modules for each of the components of the project. Seven numbered scripts run the full workflow in order, from exploring the data and building the split training, evaluation, sensitivity analysis and inspecting new images. The training model was performed using the Ultralytics 8.4.190 and PyTorch 2.4.1 libraries on a NVIDIA RTX 3050 Ti Laptop GPU (4GB). The README file in the repository includes all of the commands required to reproduce the results of the project, which can be found in Appendix A.

4.0 Alternative Solutions

To determine whether YOLOv8n was the correct choice, it was compared with four other alternatives. These alternatives range from the simplest possible rule for detecting defects to a version of the YOLOv8n model that was bigger than the one applied in the experiment. Each of these models were tested on the same set of 188 trays. The simplest alternative to the model is an ‘always FAIL’ rule that denies every tray seen. While this rule will never miss a tray with a defect, it will reject all trays that do not have a defect. The balanced accuracy of the ‘always FAIL’ rule is 0.500, setting a minimum standard that any real model must beat. Balanced accuracy is the average of defective trays found and the share of good trays that were passed, therefore it is not fooled by the fact that most trays are defective .The second alternative is a classical machine-learning baseline [7,8**]**. Each image is initially turned into a list of 2,072 numbers that describe it:

- Edge features: A Histogram of Oriented Gradients (HOG) separates the image into small cells and records the way the edges point in each one, therefore, folds, film edges and meat boundaries leave distinctive patterns.
- Colour features: A colour histogram allows the image to be identified how much of it is meat, tray or background.

A logistic regression then learns one weight per number and outputs the probability that the tray is defective. Its regularisation strength (how strongly the weights are kept small to avoid over-fitting) was tuned on the validation set. The baseline is fast, needs no GPU and is easy to interpret, but it cannot show where a defect is. The third is the project’s first attempt, YOLO11n on the official split. It scored 0.925 accuracy, but Section 5 shows that this figure was inflated by leakage and by choosing the threshold on test data. The fourth is YOLOv8s, a larger model with 11.1 million parameters (versus 3.1 million) and about 3.3 times the computation. It was trained with identical settings to test whether a bigger model helps.
In Table 2, recall is the share of defective trays caught, specificity is the share of good trays passed, and mAP50 measures how accurately individual defect boxes are found (1 = perfect).

Table 2. Comparison of alternatives on the leakage-free (grouped) test set 188 trays.

Criteria	**Always FAIL**	**HOG + colour + LogReg**	**YOLOv8n (selected)**	**YOLOv8s**
Tray recall (defective caught)	1.000	0.830	0.948	0.941
Tray specificity (good passed)	0.000	0.514	0.429	0.429
Balanced accuracy	0.500	0.672	0.688	0.685
Detection mAP50 (test)	–	–	0.508	0.423
Balanced acc. under heavy blur/noise	0.500	0.500 / 0.500	0.693 / 0.687	not tested
Localises and names defects	No	No	Yes	Yes
Inference time per image	≈ 0	5.4 ms (CPU)	9.6 ms (GPU)	10.2 ms (GPU)
Model size / compute	–	2,072 weights	3.1 M params, 8.6 GFLOPs	11.1 M params, 28.4 GFLOPs
Training cost	–	≈ 19 s	25 min (GPU)	34 min (GPU)
Interpretability	Trivial	Weights on hand-made features	Low	Low

YOLOv8n and the baseline have similar balanced accuracy on clean images, so the choice rests on three other points:

1. YOLOv8n catches more defective trays (94.8% versus 83.0%), and a miss is the costly error.
2. It stays reliable under blur and noise, while the baseline falls to the level of “always FAIL” (Section 6.3).
3. It shows what and where each defect is.

The trade-off is about twice the inference time, a GPU for training and lower interpretability. All of these are acceptable on a line inspecting a few trays per second.

YOLOv8s gave no improvement: it had the same specificity, slightly lower recall and a lower mAP50, at higher cost. This suggests performance is limited by the data (too few good trays and rare-defect examples) rather than by model size, so the smaller YOLOv8n was kept.

5.0 Validation

On video sequences that the model had never seen before, YOLOv8n was able to catch 94.8% of defective trays but only 42.9% of good trays. This official dataset split made the model look significantly better than it actually was, with a balanced accuracy of 0.873 instead of 0.688.

5.1 What was validated, and a fair test set

Three things were validated:

- The PASS / FAIL decision for each tray, since this is what the factory acts on;
- How well the detector finds and labels individual defect boxes;
- The code itself, through unit tests (Section 5.5).

Because the official test set contains near-copies of training frames (Section 2.3), a fairer grouped split was built. This follows the unit's advice that duplicates can cross the train/test boundary and that the split must match the real question being asked [9]. Here, that question is how the model performs on trays it has never seen, so whole stretches of video must be kept out of training. When it comes to building the split, these were the four things that were done:

1) All 3,549 images were sorted by frame number.
2) They were cut into separate video sequences wherever two consecutive frames were more than 20 apart, giving 117 sequences.

3) Whole sequences were randomly assigned (seed 42) to training, validation and test in a 70 / 15 / 15 ratio.
4) Only one copy of each frame was kept for validation and test.

A unit test confirms that no sequence is ever split across sets.

Table 3. Official vs. leakage-free (grouped) split.

	Official split	**Grouped split**
Train / valid / test images	3,349 / 120 / 80	2,492 / 183 / 188
Good trays in test	12	35
Exact duplicate test frames in train	4	0
Median gap, test → nearest train frame	2.5 frames	77 frames
Test frames within 5 of a train frame	67.5%	0%

The grouped test set is genuinely new footage. It is also 2.4 times larger and has about three times as many good trays, so its results are more reliable.

5.2 Metrics and Acceptance Criterion

The main metric is tray recall: the share of defective trays that are rejected. It comes first because a missed defect is a costly error. Specificity, the share of good trays that are passed, measures the cost of false rejections. Balanced accuracy is the average of the two. It is used instead of plain accuracy because 80% of trays are defective, so a model that always says FAIL would look 80% accurate while being useless; its balanced accuracy is only 0.500. For individual boxes, mAP50 measures how accurately defects are found and labelled, counting a box as correct if it overlaps the true box by at least 50% (IoU ≥ 0.5). mAP50-95 is a stricter version that averages over overlaps from 50% to 95%.

Acceptance criterion: tray recall of at least 0.95, meaning no more than 1 in 20 defective trays slips through. The threshold τ was set to the highest value that met this target on the validation set and then applied once to the test set. The baseline's settings and the early-stopping point were also chosen on validation only. The test set therefore played no part in any design choice, following the training, validation and test roles described in [10].

5.3 Results

Both YOLOv8n models were trained with the same settings. These settings include batch 8, with the training reaching 30 epochs in total, 10 in patience and seed 42. The only thing that differs was the split, therefore, any difference that comes up would be due to this. These results in differences can be seen in Table 4, below.

Table 4: Test-set results on each split.

Model	**Split**	**τ / C (from valid)**	**Tray recall**	**Specificity**	**Balanced acc.**	**Test mAP50**	**Test mAP50-95**
YOLOv8n	Official	τ = 0.25	0.912	0.833	0.873	0.443	0.212
YOLOv8n	Grouped	τ = 0.10	0.948	0.429	0.688	0.508	0.254
HOG + colour + LogReg	Official	C = 0.001	0.809	0.667	0.738	–	–
HOG + colour + LogReg	Grouped	C = 0.1	0.830	0.514	0.672	–	–

Table 5. Confusion matrix of YOLOv8n on the grouped test set (188 trays)

	Predicted FAIL	**Predicted PASS**
Actually defective (153)	145	8 (missed)
Actually good (35)	20 (false rejections)	15

The results indicate four points. First, leakage inflates the results: the official split overstates balanced accuracy by 0.19 for YOLOv8n and 0.07 for the baseline. Nearly all of this deficit is due to specificity. It misclassifies 20 of 35 good trays when it is given a new view of the trays (0.429), while on the split it correctly passes good trays it has seen correctly previously (0.833). Secondly, the acceptance criterion is just not met. Recall is at 0.948, compared to 0.95 and one more caught tray would fulfil it. Third, mAP50 would be higher on the grouped split; this is not a contradiction because mAP is averaged over the classes present and the two test sets would contain different classes, so there's no way to compare mAPs. Last but not least, it is the class that determines the performance. Defects that are not sealed score 0.911 mAP50, whereas small packaging defects (0.650) and rare classes (e.g. loose meat, 0.335) have much lower scores.

A 95% bootstrap confidence interval [11] for tray recall ranges from 0.911 to 0.980, whereas the specificity ranges between 0.265 and 0.594, due to the fact that it is based on only 35 good trays. If Ultralytics' default was set to 0.25, the balanced accuracy would be slightly higher (0.716), but 30 defective trays would be missed instead of 8. The lower threshold is hence intentionally sacrificing false rejections in exchange for less missed defects (Figure 1). As for the misclassified trays (Appendix B), there is clear evidence that most of these mistakes are on trays that lie at the edge of the frame, visible only in part, implying that the trays should be inspected as they become fully visible.

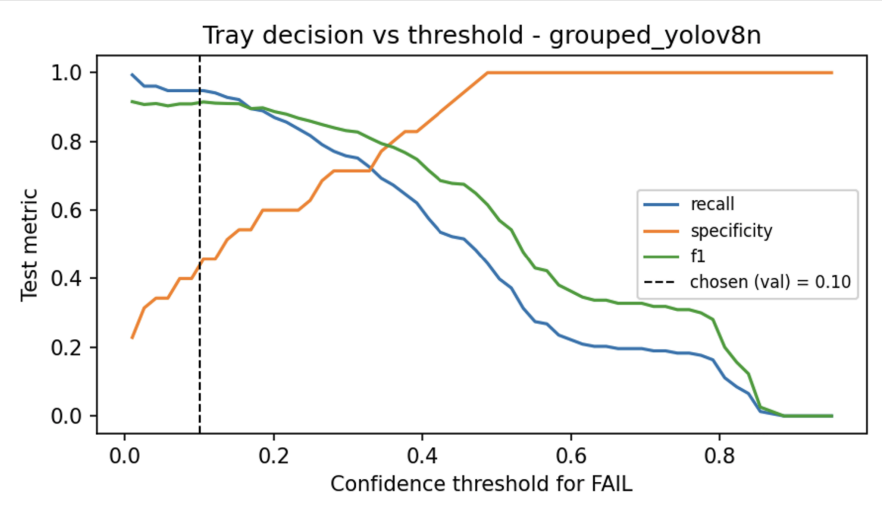


Figure 1: *Tray recall, specificity and F1 versus threshold on the grouped test set; the dashed line marks the threshold chosen on validation.*

Inspecting the misclassified trays (Appendix B) shows a clear pattern: almost every error occurs on a tray only partly in view as it enters or leaves the frame.

5.4 Engineering interpretation

If it were to be running on a real line with perhaps only 10% of trays being defective, the model would reject approximately 610 of 1,000 trays, which represents just 95 of the truly defective trays. The system would thus reduce manual inspection but not eliminate manual inspection. It is used primarily to reliably catch unsealed packs (mAP50 > 0.9) and record the type and location of each defect. The official split would have

been expected to be at 0.83 (not 0.43) and the re-checking workload would have been underestimated by about three times.

5.5 Verification of the code

41 pytest unit tests all pass (report in docs/test_report.html). They check:

- Reading labels and discarding labels that are improperly formed;
- Extracting frame numbers from file names;
- that the grouped split does not interrupt a run of data and has no leakage in a synthetic dataset; and
- The metric is used to evaluate hand calculated examples, e.g., the always-FAIL rule that yields a balanced accuracy of 0.5.

5.6 What the Validation Cannot Establish

All data is provided from one video, and it is unknown whether it will perform under any other light, camera or tray. Loose meat or wrinkles are not claimed, as they are too uncommon in test sets. Specificity, too, is unknown, with only 35 good trays.

6.0 Optimisation and Performance

6.1 Training length and early stopping

An initial 5-epoch run reached a validation mAP50 of only 0.31 and was still improving, so a 30-epoch run was performed. The mAP50 in that run increased to approximately 0.5 at epoch 21 and then stayed flat, which meant that more training wouldn't have been expected to increase mAP50. The final runs are thus at most 30 epochs with early stopping, that is stopping training when the validation score hasn't improved during 10 epochs. The grouped model peaked at epoch 17 (mAP50 0.614) and stopped at epoch 27 (training curves in Appendix B).

6.2 Input resolution: accuracy vs. cost

Larger input images preserve more detail, but require more computation and that increases by the square of the size of the image. The trained model was assessed at three different sizes with the value of τ set at 0.10 (Table 6).

Table 6. Effect of input resolution on accuracy and cost

Input size	mAP50	Tray recall	Specificity	Median latency	GFLOPs / image
320 px	0.462	0.902	0.486	8.2 ms	2.2
480 px	0.507	0.935	0.429	9.4 ms	4.8
640 px (selected)	0.508	0.948	0.429	9.3 ms	8.6

The smaller size reduces computation by a factor of 4, but reduces the recall by 4.6 points, and the size difference is only about 1 ms per image. The small time saving is due to the fact that arithmetic intensity, that is, the number of calculations per byte of data moved, is high, so YOLOv8n should be computation-limited [12]. This demonstrates that if images are processed individually, the fixed overheads like loading and transfer of images prevail. The 640 px was thus preserved.

6.3 Robustness to realistic disturbances

The test images were degraded with brightness changes, motion blur, sensor noise and glare, and both models were re-scored without re-training, to simulate realistic line conditions (Figure 2). This tests extrapolation beyond the training conditions [13].

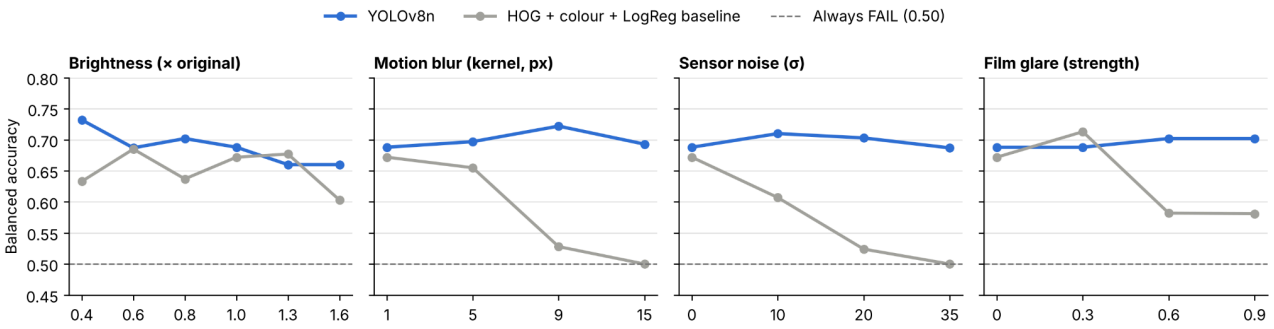


Figure 2: *Balanced accuracy for image disturbances, grouped test set (188 trays), τ = 0.10.*

YOLOv8n maintained its accuracy between 0.66 and 0.73 under all disturbances. The baseline, however, dropped to 0.500, the "always FAIL" level, with a lot of blur and noise. These disturbances destroy its edge-designed features, while those learned by YOLOv8n with augmentation remain after the disturbances.

6.4 Speed, memory and training cost

YOLOv8n achieves a median speed of 9.6 ms/im on the GPU and approximately 100 trays/sec, whereas the baseline is 5.4 ms/im on the CPU. With the baseline it takes seconds, while training takes 25 minutes with GPU memory of 1.23 GB. The training speed was largely dependent on the data load, taking 26 s per epoch with batch 16 and 2 loader workers, compared to 53 s per epoch with batch 8 and no loader workers. Both final runs were made at slower settings ensuring split was the only variable. The speed the production line requires (full table in Appendix C) is met easily by both models.

7.0 Limitations, Lessons Learned and Improvements

7.1 Limitations

The major drawback is the high false-rejection rate. The model also rejects 57% of good trays (20 of 35); these would all need manual re-checking. There was no threshold that had both high recall and specificity values above 0.5. The decision rule is straightforward: a tray fails on its single most confident detection, so one spurious box can reject a good tray. Rare defects are not supported. Wrinkle, loose meat and twisted meat have too few examples to learn or evaluate reliably, so the system should be described as detecting mainly unsealed and packaging-error defects. The evaluation sets are small, so with only 35 good test trays the confidence intervals are wide. In addition, all frames come from one camera and one video, so any new line would need re-validation. Finally, each configuration was trained only once, so the variation between runs is unknown.

7.2 Lessons learned

The most important lesson is that a high score is not evidence until the test set is shown to be independent. The first model's 0.925 accuracy was largely due to near duplicate frames, and by looking at the frame numbers, which took just a few minutes, it changed the project's conclusions [9].

There were three lessons to be learned from the process of trial and error:

- Tune not on the test set. The first threshold was chosen by looking at test results; moving this choice to the validation set made the reported figures honest.
- Vary the parameters one at a time. The first 2 training runs were different in settings and split, therefore they were re-trained with identical settings.

- Choose metrics that align with the problem. WIth 80% defective trays, plain accuracy rewards rejecting everything, and mAP depends on which classes the test set contains; balanced accuracy at a fixed threshold gives a fair comparison.

7.3 Proposed improvements

Better and increased amounts of data, more good trays and with the few defects noted across multiple sessions, all the classes are able to be tested.
Reduced false rejections (per class thresholds, minimum box size, or high resolution for small packaging errors). Checking each tray when it is completely visible, or making predictions from multiple frames on the same tray (due to the fact that most errors were on partially visible trays).
Improved evaluation by grouped cross-validation over video sequences to assess the run-to-run variation, with a pilot test on a live line.

8.0 Conclusions

This project developed a YOLOv8n-based inspection pipeline that detects packaging defects on pork-rasher trays and makes a PASS / FAIL decision for each one. Its most important finding concerns validation rather than the model. The official dataset split leaks near-duplicate video frames into its test set, which inflated balanced accuracy from 0.688 to 0.873. A grouped split by video sequence gave the realistic estimate. On genuinely unseen footage, the detector caught 94.8% of defective trays, just short of the 0.95 target. It also outperformed the classical baseline in recall and robustness to blur, noise and glare, and a larger YOLOv8s model gave no further gain. However, it rejected more than half of the good trays. The system is therefore best suited as a decision-support tool that flags suspect trays for an operator, not as a replacement for manual inspection. More balanced data and inspecting each tray only when it is fully in view are the clearest next steps.

9.0 AI Use and Reflection

I used Claude (Anthropic, via claude.ai) throughout this project. I chose the dataset and problem and directed the work by prompting Claude step by step to help initialise, debug, improve and explain the code. For the report, Claude drafted section text from my results, which I edited and corrected, and it re-plotted Figure 2 from my results file. AI contributed most of the code and a good amount for the first-draft wording; the experiments, checks and decisions were mine, where I reworded the report myself. I used AI because this was my first object-detection project and time was limited.

I verified everything by running all scripts and 41 unit tests on my laptop and checking every reported number against my output files. This caught several AI errors: a crashing Matplotlib argument, a FLOP counter returning no value, Windows crashes (fixed with workers=0), mismatched training settings (retrained), and reworded sentences with wrong meanings or numbers.

My key decisions were making the leakage-free split the headline result, prioritising caught defects, keeping YOLOv8n at 640 px, and reporting the borderline recall honestly. AI let me attempt a larger project, but taught me that its output must always be checked.

References

[1] G. Jocher, A. Chaurasia and J. Qiu, “Ultralytics YOLOv8,” version 8, 2023. https://github.com/ultralytics/ultralytics
[2] Roboflow user hello-2aqe0, “Pork Rasher Error (Packaging)” dataset, version 4, Roboflow Universe, 2025. Licence CC BY 4.0. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging
[3] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, “Leakage in Data Mining: Formulation, Detection, and Avoidance,” ACM Trans. Knowl. Discov. Data, vol. 6, no. 4, 2012.
[4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE CVPR, 2016, pp. 779–788.
[5] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. ECCV, 2014, pp. 740–755.
[6] MMA3001 Unit Notes, “5.4 Neural Network Regression,” Monash University, 2026.
[7] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” in Proc. IEEE CVPR, 2005, pp. 886–893.
[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
[9] MMA3001 Unit Notes, “5.6 Training Data Evaluation and Selection,” Monash University, 2026.
[10] MMA3001 Unit Notes, “5.5 Evaluating Regression Performance,” Monash University, 2026.
[11] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” Ann. Statist., vol. 7, no. 1, pp. 1–26, 1979.
[12] MMA3001 Unit Notes, “2.1 Optimisation, Profiling and Machine Performance,” Monash University, 2026.
[13] MMA3001 Unit Notes, “3.6 Interpolation in Machine Learning,” Monash University, 2026.
[14] MMA3001 Unit Notes, “1.4 Responsible Use of Artificial Intelligence,” Monash University, 2026.
[15] Anthropic, Claude (Opus model), large language model, accessed 27–29 October 2026. https://claude.ai

Appendix A: Reproducing the Results

The code is available at https://github.com/zsab0002/MMA3001-Zhafira-Sabrina-Project. With the dataset unzipped into data/raw/ and the dependencies installed (pip install -r requirements.txt, then pip install -e .), the scripts are run in this order:

1. python scripts/01_explore_data.py
2. python scripts/02_make_grouped_split.py
3. python scripts/03_baseline.py
4. python scripts/04_train_yolo.py --split grouped and python scripts/04_train_yolo.py --split original
5. python scripts/05_evaluate.py
6. python scripts/06_sensitivity.py --run grouped_yolov8n
7. python scripts/07_inspect.py --source data/grouped/test/images --run grouped_yolov8n
8. pytest --html=docs/test_report.html --self-contained-html and pdoc src/porkqc -o docs/api --docformat google

Environment: Windows 11, Python 3.10.5, PyTorch 2.4.1 + CUDA 11.8, Ultralytics 8.4.190, NVIDIA RTX 3050 Ti Laptop GPU (4 GB). Results may vary slightly (about ±0.01 mAP) on other hardware. A GPU is not required; the code also runs on a CPU, more slowly.

Appendix B: Supporting figures

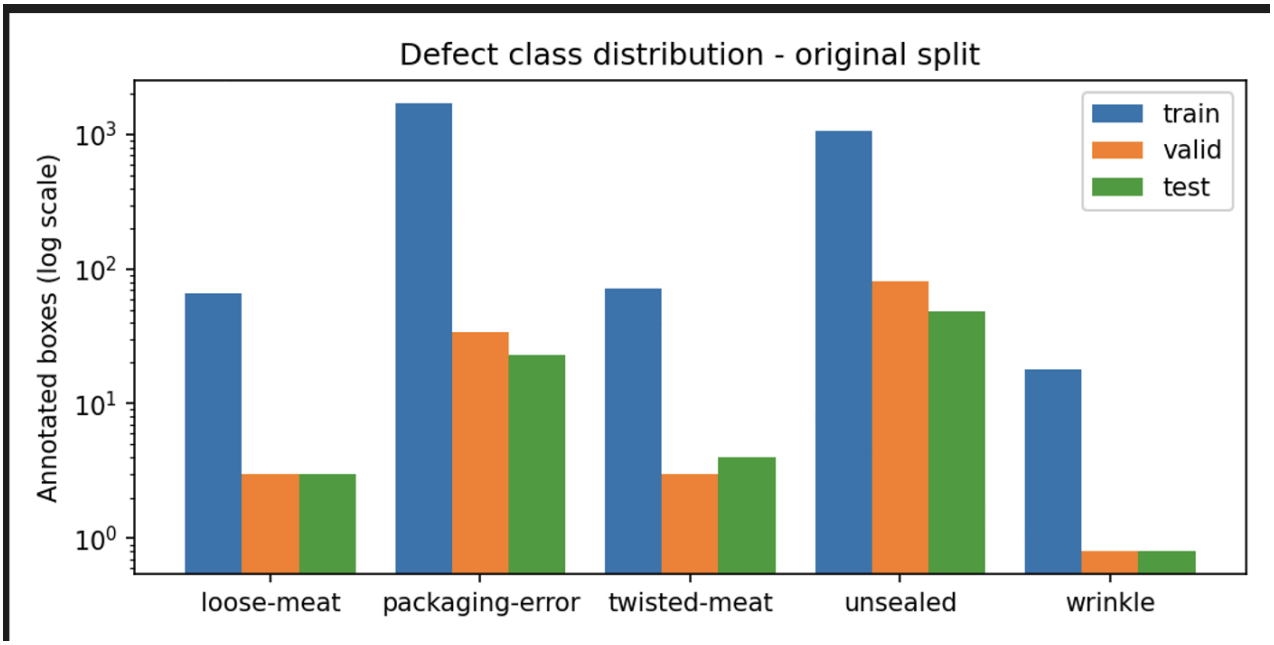


Figure B1: Defect boxes per class and split (log scale).

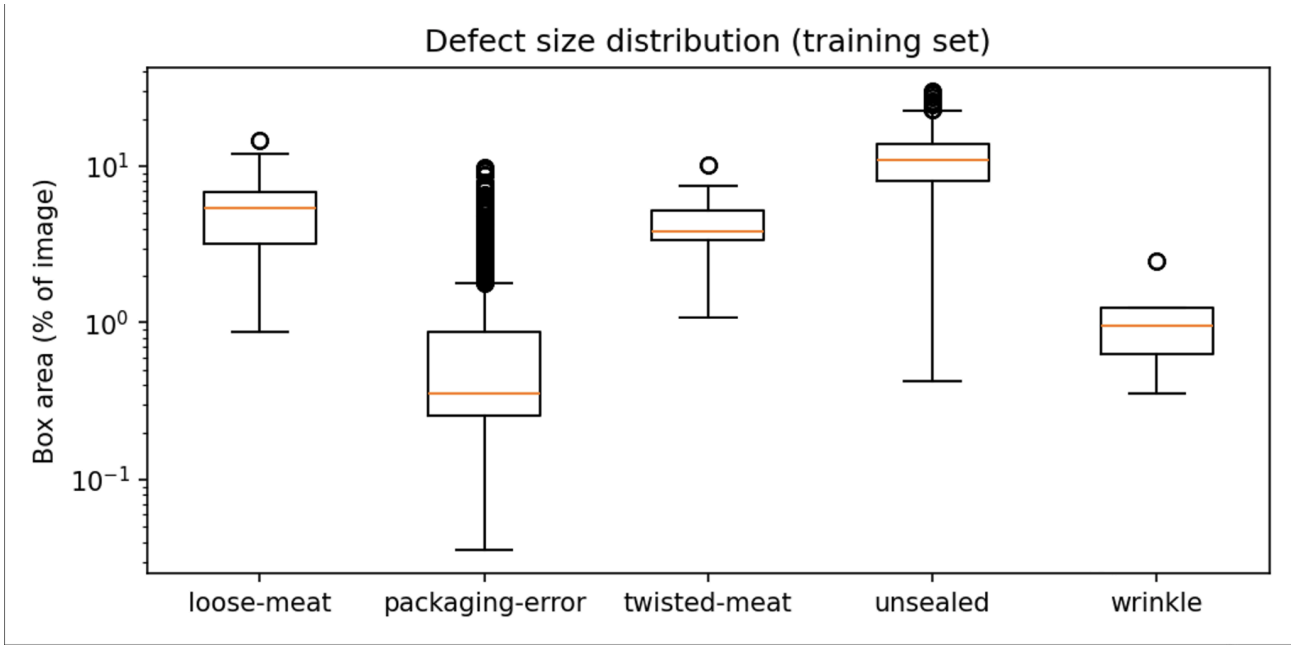


Figure B2: Defect box area as a percentage of the image, training set (log scale).

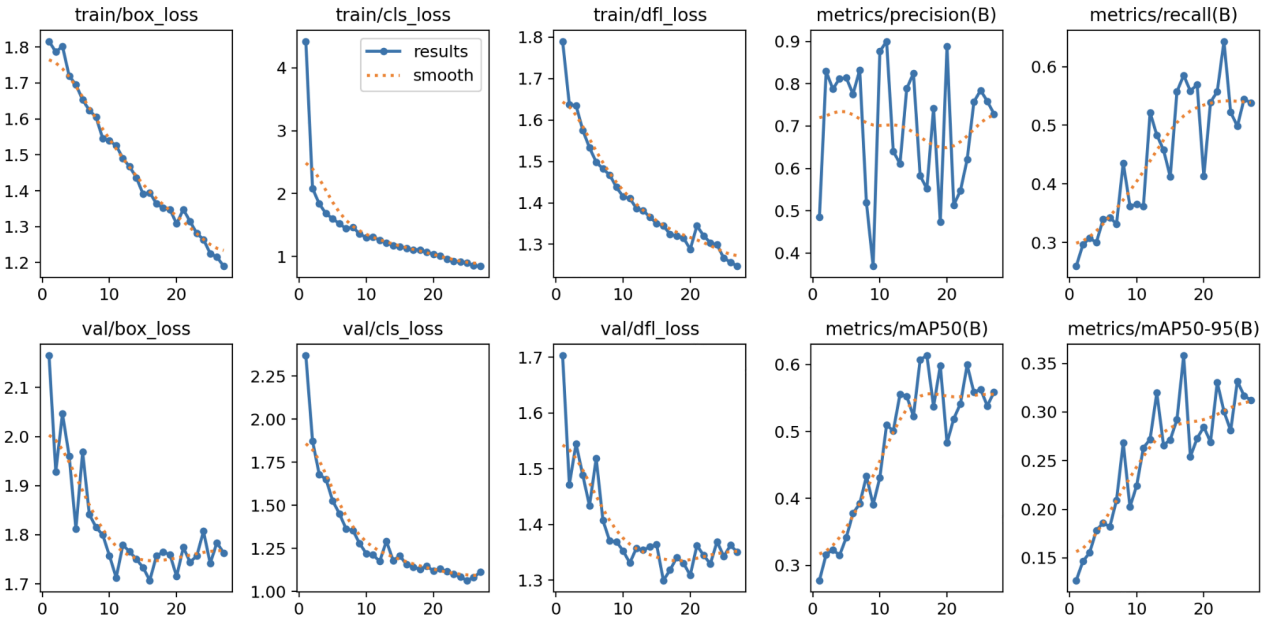


Figure B3: Training and validation curves of the grouped YOLOv8n model.

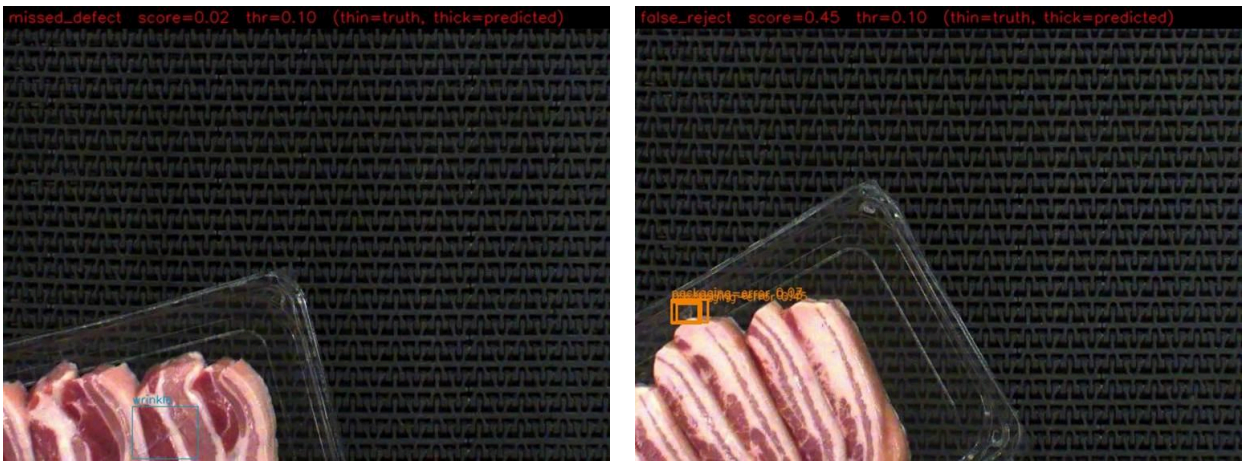


Figure B4: Failure modes on partly visible trays: a missed wrinkle defect (left) and a false rejection from a small packaging-error prediction at the tray corner (right). Thin boxes = ground truth, thick boxes = prediction.

Appendix C: Specification and data tables

Table C1: Top-level inputs and outputs of the inspection system.

Item	Format and units	Domain	Meaning and use
Input image	JPG/PNG, RGB, 8-bit per channel (0–255)	720 × 540 px, top-down view of one tray	Tray to inspect. Other sizes are accepted but flagged as outside the validated domain.
Detections	List of (class, confidence, x1, y1, x2, y2)	Class ∈ {loose-meat, packaging-error, twisted-meat, unsealed, wrinkle}; confidence 0–1; corners in pixels	What each defect is and where, for the operator.
Decision	PASS / FAIL / INVALID	FAIL if any confidence ≥ τ (τ = 0.10, tuned on validation)	Whether the tray is rejected for manual inspection.
Severity	none / minor / major / critical	Box area ÷ image area: < 1% minor, 1–5% major, > 5% critical	Prioritises which rejected trays to check first.
Report	CSV + JSON, plus annotated image	–	Audit trail of every decision.

Table C2: Composition of the official Roboflow split.

Split	Images	Defective / good	Boxes: packaging-error / unsealed / twisted / loose / wrinkle
Train	3,349	2,665 / 684	1,712 / 1,055 / 72 / 66 / 18
Valid	120	106 / 14	34 / 81 / 3 / 3 / 0
Test	80	68 / 12	23 / 49 / 4 / 3 / 0

Table C3: Model and decision settings (all held in configs/config.yaml).

Setting	Value	Reason
Model	YOLOv8n, COCO pre-trained	Smallest, fastest variant; YOLOv8s compared in Section 4
Input size	640 px	Small packaging errors need resolution (Section 6.2)
Max epochs / early stopping	30 / patience 10	A first 30-epoch run levelled off by about epoch 21
Batch size	8	Fits the 4 GB GPU with margin
Augmentation	Ultralytics defaults (flips, colour jitter, scaling; mosaic off for last 10 epochs)	Reduces over-fitting
Seed	42, deterministic mode	Reproducibility
Decision threshold τ	0.10	Highest value keeping validation tray recall ≥ 0.95

Table C4: Speed, memory and computational cost.

Measure	HOG + colour + LogReg	YOLOv8n
Inference per image	5.4 ms (CPU)	9.6 ms median (GPU); 2.9 ms network only
Throughput	≈ 185 images/s	≈ 100 trays/s
Peak GPU memory, training	–	1.23 GB at batch 8 (YOLOv8s: 1.92 GB)
Compute per image	≈ 2,072 multiply-adds after feature extraction	8.6 GFLOPs at 640 px
Training time	≈ 19 s	25 min (grouped split), 37 min (official split)